

Spartan Crew — Enquiry → Booking Engine

Rebuild plan & prototype status · prepared 2026-07-13 · **supersedes** Spartan-Crew-Vercel-Port-Handoff.pdf

What changed since the last brief. The June port brief planned a *strangler-fig port* — keep n8n as the orchestrator and swap nodes for Vercel functions one at a time. The vision doc (Automate the Enquiry to booking process) asks for something stronger: a **from-scratch rebuild** around a **conversational renderer/compiler + a Save State Table**, deterministic and hands-free. Confirmed client-facing scope is a **metrics dashboard + a settings menu, and nothing else**. This document is that rebuild plan. The port brief's research (live-workflow map, OnSinch API contract) is still valid input and is folded in below.

1. The one idea

The current system is **155 n8n nodes across 3 workflows** because n8n forces you to express *state reconciliation* as *dataflow*. Every relationship — this thread → this order, this company name → this id — is re-derived from scratch on every email, wired by hand.

The fix the vision names — "conversational renderer + Save State Table" — is to make the **Gmail thread the unit of work**, hold its canonical state in **one table row**, and **compile** that row into the next actions. The entire system collapses to one idempotent loop:

```
on any Gmail event (poll OR push):
  thread  = gmail.hydrateThread(threadId)    // always the FULL conversation
  prior   = store.get(threadId)              // the Save State row (or none)
  {state, actions} = compile(thread, prior)  // READS only; derives desired state
  execute(actions)                          // reply draft + OnSinch create/patch
  store.put(state)                          // persist (idempotent)
```

Because `compile()` only reads and is keyed on `thread_id`, running it again on any thread is harmless. **That is the "never miss a lead" guarantee:** a nightly sweep can re-compile every thread in the inbox and produce zero duplicate orders or drafts. Missed poll windows, crashes mid-run, and late-arriving mail all self-heal.

2. Why a rebuild does the job better

Problem in the live n8n system	Fix in the engine
155 fragile nodes ; every node a failure point, near-impossible to change safely	One <code>compile()</code> function + small typed I/O clients. Logic is code you can read, test, and diff.
10-minute poll window → a boundary failure silently drops a lead	Thread-keyed state row + idempotent recompute. Nightly full-sweep backfills anything missed.
The <code>stream:query:data</code> verbatim-extraction hack (25+ labels reconciled by hand)	One structured extraction → a typed <code>ConversationFacts</code> object.
Compose agent emits flat <code>key: value</code> text lines, re-parsed by another node — brittle text protocol	A typed <code>DesiredOrder</code> ; the body builder is a pure, total function.
Dedup by fuzzy Airtable lookup on every run	The state row is the dedup key: <code>thread_id</code> → <code>onsinch_order_id</code> .
LLM resolves integer ids (company/user/place) — non-deterministic where it must be exact	Ids resolved in deterministic code + local scoring. The model never emits an id.
Black box in n8n — no visibility for the client	Vercel dashboard reads the state table directly; every decision is a row.
Four models (gemini-flash, gemini-pro, glm-5, gpt-5-nano)	One model, <code>claude-opus-4-8</code> , temperature 0, for the 3 language tasks only.

3. What the live system does today (folded-in research)

Three **active** workflows (6 others are archived/dead):

- **Bookings v3.4** (45 nodes) — orchestrator. Gmail poll → normalize → two branches: (a) *reply*: draft an LLM reply, label HIGH/MED/LOW, create a Gmail draft with Message-ID threading; (b) *job*: classify "is this an order?", log to Airtable, call v1.2.
- **Bookings v1.2 Job Automation** (70 nodes) — the OnSinch order engine. Extract facts → resolve `company_id` / `place_id` / `user_id` → compose order → `POST/PATCH /orders` → log to Airtable + Sheets.
- **Mini Operations** (40 nodes) — reply-only mirror of the Bookings reply branch (no job branch).

Operations and Bookings already share an identical reply architecture, so the engine has **one reply path** parameterised by inbox, plus a **Bookings-only order path**. All current prompts and deterministic code (Normalize Data, Score Findings, format-for-OnSinch) are captured and ported.

4. The Save State Table

One row per Gmail thread. This is the dedup key, the idempotency anchor, and the dashboard's data source.

```
ConversationState {
  thread_id           // PRIMARY KEY – the Gmail thread
  subject, participants[], last_message_id, last_processed_epoch

  classification      // new-job | update | confirmation-only | not-a-job
  facts               // typed ConversationFacts (replaces stream:query:data)

  company_id?, user_id?, place_id?      // resolved once, then cached – never re-guessed
  onsinch_order_id?, onsinch_order_number? // this thread's order (the dedup link)

  desired_order       // DesiredOrder | null   (null = info-only)
  last_ordered_hash?  // hash of the order we last SENT – makes patch idempotent

  priority            // low | medium | high
  reply_subject?, reply_body_html?, reply_draft_id?, last_reply_hash?

  needs_human         // confidence gate for hands-free operation
  status              // open | drafted | ordered | error | ignored
  notes[], order_action_log[]          // full audit trail for the dashboard
}
```

5. Deterministic vs. LLM — the discipline

Rule: the model reads and writes prose; it never resolves an id or builds the order body. Exactly **three** tasks touch a model.

Step	Kind	Notes
normalize thread	code	clean bodies, strip Spartan sigs/quotes, chronological, drop dupes
classify latest email	LLM	new-job / update / confirmation-only / not-a-job + priority
extract facts	LLM	→ typed <code>ConversationFacts</code> ; verbatim values, dates → YYYY-MM-DD
resolve company_id / user_id	code + API	search OnSinch, exact-match, cache; reuse prior row's id
resolve place_id	code + API	OnSinch has no fuzzy search → local <code>Score Findings</code> port; ≥75% or create-new
compose DesiredOrder	code	business rules in code: <code>request_approval:true</code> , place_id on every slot, 08:00–18:00 defaults, profession mapping, crew-chief flag
compose reply	LLM	casual crew tone, HTML body, never leaks IDs/priority/internal ops
validate + diff + dedup	code	pre-flight the 400-causers; hash-diff to decide create vs patch vs no-op

6. The compiler, step by step

1. **Normalize** the thread → `{latest, history}` .
2. **Classify** the latest email against the thread + prior state.
3. **Compose reply** (every inbound gets a draft; hash-diffed so re-runs don't re-draft).
4. If it's a job: **extract facts**, then **resolve** company/place/user ids (reusing cached ids from the row).
5. **Compose** the `DesiredOrder` and **validate** it.
6. **Confidence gate**: unresolved id or validation error → `needs_human = true` , order is held for review — the reply still drafts.
7. **Diff to actions**: no order id yet → `createOrder` ; order exists and desired changed → `patchOrder` ; unchanged → nothing.

The compiler performs only reads (GETs are safe/idempotent). Writes (`createOrder` , `patchOrder` , `createReplyDraft`) are **returned as an Actions object** for a separate executor — keeping compile pure enough to re-run anytime.

7. Pipeline (confirmed scope)

The client-confirmed structure is a five-stage funnel; each stage is a compiler step, and each emits one metric event:

Stage	Engine step	Metric emitted
reliably intake every email	thread-keyed row + idempotent recompute + nightly sweep	<code>email_received</code> , <code>thread_processed</code>
filter / render	<code>normalize</code> + <code>classify</code> (not-a-job → ignored)	<code>filtered_out</code>
extract client job requests	<code>extractFacts</code> → typed <code>ConversationFacts</code>	<code>job_detected</code>
generate reply	<code>composeReply</code> → threaded draft	<code>reply_drafted</code>
create the OnSinch job	resolve ids → compose → validate → create/patch	<code>order_created</code> / <code>order_updated</code> / <code>order_error</code>

8. The Vercel app — dashboard + settings, that's it

The **only** client-facing surfaces are a dashboard and a settings menu. Architecture mirrors the House of Hud quote tool + its metrics dashboard.

Metrics (mirrors the HoH dashboard)

An append-only `metric_events` table (Neon/Vercel Postgres, exactly like HoH) records one row per pipeline transition. The dashboard **never** queries live Gmail/OnSinch — it aggregates this log only. Headline tiles + funnel:

- **Headline tiles:** hours saved, orders created, clients served, awaiting confirmation, hands-free rate, needs-human count, order-error rate.
- **Funnel:** emails received → threads processed → filtered out → job requests → replies drafted → orders proposed → created/updated.
- **Confirm queue:** in draft-only mode, every proposed order sits here with its resolved company/place/crew for **one-click approve** (calls `confirmOrder`) — the human-in-the-loop before anything is written to OnSinch.
- **Queue view:** the state table as a live list — status, priority, order links, and a **"needs human"** lane for anything the confidence gate held back.

Settings

- **order_mode: draft-only (launch default) vs. auto**, confidence threshold, model, block-list, inbox creds, minutes-saved-per-email tuning.

The engine runs as Vercel Functions (compile + executor) triggered by Gmail push (Pub/Sub) with a cron sweep as backstop. State + metrics live in Postgres. Git-versioned throughout. *(The helper chatbot from the original vision is deferred — current scope is dashboard + settings only.)*

9. OnSinch contract — the essentials that bite

Full reference: `Spartan-Crew-Onsinch-API-Reference.md`. The rules the engine must respect:

- Auth header is literally `Authorization: apikey <KEY>` — **not Bearer**.
- Every write is a **JSON array**, even for one item. `POST /orders` nests `Job` (object) + `SlotTeam[]`.
- `PATCH` returns **204 no body** — don't parse JSON.
- `place_id` is **required on every SlotTeam** — the single most common 400.
- Creating a Place requires `country` (ISO-2). No fuzzy search anywhere — that's why local scoring exists.
- `profession_id`: 1 Crew (default) · 32 CSCS · 9 Driver · 16 AV · 36 Crew Chief.

10. Data studies the vision calls for (planned)

- **Writing-style study** — mine 2000+ sent Spartan emails to build the reply prompt's tone/edge-case guide (same corpus tooling used for the HoH mail study).
- **OnSinch edge-case map** — a "neural map" of quoting rules and failure modes from the API + real orders, driving the deterministic composer (esp. the crew-chief rule, TBC dates, multi-block splits).

11. Prototype — built & proven today DONE

Location: D:\code\SpartanCrew-Enquiry-Engine. The **foundational core** is written, typechecks clean, and passes an offline end-to-end test (no network).

Module	Status
<code>compiler.ts</code> — the <code>compile()</code> loop	built
<code>types.ts</code> — <code>ConversationState</code> / <code>DesiredOrder</code> / facts	built
<code>normalize.ts</code> , <code>score.ts</code> , <code>format.ts</code> — deterministic ports of the live n8n code	built
<code>compose.ts</code> — facts + ids → <code>DesiredOrder</code> , business rules in code	built
<code>onsinch.ts</code> — typed client (injectable transport)	built
<code>reason.ts</code> — 3-task LLM boundary + Anthropic (Opus 4.8) adapter	built
<code>store.ts</code> — Save State Table interface (in-memory; Postgres/KV later)	built
<code>metrics.ts</code> — append-only <code>metric_events</code> + dashboard aggregate (HoH pattern)	built
<code>pipeline.ts</code> — per-event handler: compile → execute → emit metrics	built

Test proves (20/20 assertions, offline): draft-only new-job → reply drafted + order **proposed, not written** (ids resolved, `place_id` on the slot); **one-click confirm** → order created; re-handling the thread → no second order (idempotent); follow-up crew-count change → proposed **PATCH** → confirm; bare "thanks" → confirmation-only; the **dashboard aggregate** reflects the funnel; and flipping `order_mode: "auto"` writes hands-free. Run with `npm test`.

12. Build roadmap

1. **Phase 0 (done)** — foundational engine + offline proof.
2. **Phase 1** — real Gmail hydrate/draft client; Postgres/KV store; wire full ported prompts into the Opus reasoner; run against a replay of real threads.
3. **Phase 2** — Vercel app shell: metrics dashboard (funnel + tiles) + settings + the confirm queue + needs-human lane. **Launches in draft-only mode** — every order human-approved.
4. **Phase 3** — Gmail push trigger + cron sweep; flip `order_mode` to auto once the needs-human/error rate is proven low.
5. **Phase 4** — data studies feed the style guide + edge-case map.

13. Open decisions

- **Auto vs. draft-only at launch** — **DECIDED: draft-only**. Every order is human-approved via the confirm queue until the needs-human/error rate is measured; then flip `order_mode` to auto.
- **State store** — Vercel Postgres (relational, dashboard-friendly) vs. KV. Recommend Postgres.
- **Trigger** — Gmail push (Pub/Sub) vs. cron poll. Recommend push + a nightly cron sweep as the idempotent safety net.
- **Crew-chief rule semantics** — does ≥ 4 crew split a slot or just tag one? The composer flags it today; the edge-case study must settle it.

Prototype: D:\code\SpartanCrew-Enquiry-Engine · OnSinch contract: Spartan-Crew-Onsinch-API-Reference.md · Fresh n8n export: D:\Business\Samurai Solutions\n8n_export_fresh · Vision: Google Doc "Automate the Enquiry to booking process". Supersedes the June 2026 Vercel-Port handoff.